Algorithmique Fondamentale et Structures de Contrôle

Thème principal : Bases de l'algorithmique impérative : variables, typage, alternatives et itérations

Synthèse exécutive : Ce cours formalise la démarche intellectuelle et technique permettant de traduire un problème du monde réel en une séquence finie d'instructions exécutables par un ordinateur. Il explore l'organisation de la mémoire à travers le typage strict des variables et l'opération d'affectation destructrice. Il traite ensuite de la prise de décision via l'algèbre booléenne et l'évaluation court-circuit, avant de développer la conception rigoureuse des boucles (Pour et Tant Que), en mettant l'accent sur la preuve de terminaison par variant de boucle et la gestion robuste des états sentinelles pour prévenir les plantages à l'exécution.

Fondements Algorithmiques, Organisation Mémoire et Typage des Données

L'ordinateur ne possède pas d'intelligence propre ; il exécute aveuglément des instructions codées en binaire. L'algorithmique constitue l'étape amont de conception formelle indépendante de tout langage de programmation spécifique. Avant d'exécuter un traitement, la machine doit réserver un espace d'adressage en mémoire centrale (RAM). Le 'type' d'une donnée définit précisément le nombre d'octets alloués à la case mémoire ainsi que l'ensemble des opérations arithmétiques ou logiques licites sur cette donnée. L'opération d'affectation constitue l'acte élémentaire de modification de l'état mémoire : elle évalue d'abord la valeur produite par le membre droit avant de l'écrire dans la cellule identifiée à gauche. Toute valeur précédemment stockée dans cette cellule est alors définitivement perdue.

- Définition d'un algorithme : suite finie, déterministe et non ambiguë d'instructions élémentaires opérant sur des données pour produire un résultat déterminé.
- Composants d'une variable : identificateur (nom symbolique), adresse physique en mémoire vive (RAM), type de données et valeur courante.
- Types primitifs fondamentaux : Entier (Integer), Réel à virgule flottante (Float), Booléen (Boolean) et Chaîne de caractères (String).
- Affectation destructrice : opération (notée '<-' ou '=') qui évalue l'expression à droite et écrase irrémédiablement le contenu de la case mémoire cible à gauche, à distinguer rigoureusement du test d'égalité relationnelle ('==').

Cas concret enseignant : *La métaphore de la boîte aux lettres normalisée : une variable agit comme une boîte aux lettres portant une étiquette (l'identificateur) et dotée d'une fente aux dimensions fixes (le type). On ne peut pas introduire un carton volumineux (chaîne de caractères ou réel double précision) dans une fente prévue pour une fine carte postale (entier court ou booléen). Lorsque le facteur dépose un nouveau courrier dans la boîte, il détruit et remplace instantanément le courrier précédent qui s'y trouvait.*

Prise de Décision : Algèbre Booléenne et Évaluation Court-Circuit

Pour qu'un programme résolve des cas réels, son parcours d'exécution doit pouvoir bifurquer en fonction de l'état dynamique de ses variables. Les structures de contrôle conditionnelles s'appuient sur l'évaluation de prédicats combinant des opérateurs relationnels (<, <=, >, >=, ==, !=) et logiques. Un concept fondamental enseigné en IUT est l'évaluation dite 'court-circuit' (short-circuit evaluation). Dans une conjonction de type 'A ET B', si l'opérande A s'évalue à Faux, l'expression entière sera nécessairement Fausse, quelle que soit la valeur de B : l'ordinateur s'abstient donc d'évaluer B. Inversement, pour 'A OU B', si A est Vrai, B n'est pas évalué car l'expression globale est déjà acquise. Cette propriété est vitale en programmation avancée pour neutraliser les erreurs d'exécution fatales, comme tester qu'un pointeur ou une référence n'est pas nulle avant de lire son contenu, ou vérifier qu'un indice de tableau reste dans les bornes autorisées avant d'accéder à l'élément.

- Structure alternative : blocs 'Si (condition) Alors ... SinonSi ... Sinon ... FinSi' permettant de dévier le flux linéaire d'instructions.
- Prédicat booléen : expression logique dont le résultat est strictement binaire (Vrai ou Faux).

• Opérateurs logiques et lois de De Morgan : manipulation des connecteurs ET (conjonction), OU (disjonction) et NON (négation).

• Principe de court-circuit : stratégie d'optimisation où l'évaluation des conditions s'arrête dès que le résultat final de l'expression globale est logiquement certain.

Cas concret enseignant : *Le portillon d'accès au réseau de transport d'Anglet/Bayonne : pour valider le passage, le système vérifie la condition '(Titre_Valide ET Solde_Suffisant)'. Si le titre présenté est expiré (Titre_Valide = Faux), le tourniquet bloque immédiatement l'usager sans prendre la peine d'interroger le serveur bancaire central pour contrôler le solde. Cette coupure précoce économise de la bande passante réseau et évite d'éventuels temps d'attente inutiles.*

Structures Répétitives : Boucles Déterminées et Indéterminées

L'automatisation du traitement de gros volumes d'informations repose sur la répétition contrôlée d'instructions. Le choix de la structure de contrôle appropriée obéit à une règle d'ingénierie stricte : si le nombre d'itérations est fixé ou calculable avant le démarrage du bloc répété, l'utilisation de la boucle 'Pour' est impérative, car elle encapsule automatiquement l'initialisation, le test de borne et l'incrémentation. Lorsque le critère d'arrêt est imprévisible (attente d'une saisie utilisateur conforme, lecture de fin de fichier, franchissement d'un seuil de convergence numérique), la boucle 'Tant Que' s'impose. La boucle 'Tant Que' présente un danger majeur : l'infinisation (boucle infinie). Pour prouver qu'une boucle 'Tant Que' termine, le concepteur doit prouver l'existence d'un variant de boucle, garantissant mathématiquement que la condition de continuation deviendra inévitablement fausse après un nombre fini d'étapes.

• Boucle bornée (Pour / For) : structure itérative déterministe où le nombre total d'itérations est connu avant d'entrer dans la boucle.

• Boucle non bornée (Tant Que / While) : répétition assujettie à une condition booléenne d'arrêt, indispensable lorsque la fin dépend d'un événement externe ou d'un calcul itératif convergent.

• Variant de boucle : quantité entière strictement positive qui décroît à chaque itération et garantit la terminaison de l'algorithme.

• Invariant de boucle : propriété logique restant vraie avant, pendant et après chaque itération, utilisée pour démontrer la correction d'un algorithme.

Cas concret enseignant : *Le distributeur automatique de billets de banque : l'automate ne sait pas d'avance combien de billets il doit délivrer au client car cela dépend de la somme réclamée et des coupures restantes dans ses bacs physiques. Il applique un 'Tant Que (Somme_Restante > 0)', en sélectionnant à chaque itération le plus gros billet disponible inférieur ou égal à la somme restante, puis en soustrayant cette valeur. La quantité 'Somme_Restante' agit exactement comme le variant de boucle qui décroît vers 0.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne pas toujours employer une boucle 'Tant Que' dans nos programmes, sachant qu'elle peut techniquement simuler n'importe quelle boucle 'Pour' ?
Réponse du professeur : C'est une question de clarté algorithmique, de maintenabilité et de prévention des bogues. Une boucle 'Pour' matérialise immédiatement l'intention que le nombre d'itérations est fixé à l'avance et gère mécaniquement l'incrémentation de la variable d'indice. Remplacer un 'Pour' par un 'Tant Que' oblige à gérer manuellement l'incrémentation du compteur en fin de corps de boucle ; un simple oubli ou une instruction 'continuer' mal placée engendre une boucle infinie. L'usage d'un 'Tant Que' là où un 'Pour' était naturel est sanctionné en contrôle continu.

Question étudiant : Quelle est la différence fondamentale entre l'opérateur 'OU' logique standard et l'opérateur 'OU EXCLUSIF' (XOR) ?
Réponse du professeur : Le OU logique standard en algorithmique est inclusif : l'expression 'A OU B' est Vraie si A est vrai, si B est vrai, ou si les deux conditions A et B sont simultanément vraies. À l'inverse, le OU exclusif (XOR) exige qu'une seule et unique condition soit vraie : si A et B sont tous les deux vrais en même temps, 'A XOR B' s'évalue à Faux.

■ Exercice & Démonstration d'Application

Énoncé : Concevoir un algorithme robuste qui demande à l'utilisateur de saisir une suite de notes réelles strictement positives et calcule leur moyenne arithmétique. La saisie s'interrompt dès que l'utilisateur entre la valeur sentinelle -1. Si l'utilisateur saisit des notes négatives aberrantes (différentes de -1), elles doivent être ignorées avec un message d'alerte. Si aucune note strictement positive valide n'a été saisie avant l'arrêt, l'algorithme doit avertir l'utilisateur sans provoquer d'erreur fatale d'exécution.

Résolution pas à pas :
Étape 1 : Déclaration et initialisation des accumulateurs. somme <- 0.0 (Réel pour cumuler les notes) nb_notes <- 0 (Entier pour compter les notes valides) Étape 2 : Lecture anticipée (amorce de la boucle sentinelle). Afficher 'Veuillez saisir une note (-1 pour terminer) : ' Lire note Étape 3 : Structure itérative conditionnelle conditionnée par la sentinelle. Tant Que (note != -1.0) Faire Si (note > 0.0) Alors somme <- somme + note nb_notes <- nb_notes + 1 Sinon Afficher 'Avertissement : Note négative invalide ignorée.' FinSi Afficher 'Veuillez saisir une note (-1 pour terminer) : ' Lire note FinTantQue Étape 4 : Traitement post-boucle et sécurisation arithmétique. Si (nb_notes > 0) Alors moyenne <- somme / nb_notes Afficher 'La moyenne des notes valides est : ', moyenne Sinon Afficher 'Erreur : Aucune note strictement positive n'a été enregistrée.' FinSi

Résultat : L'algorithme traite correctement la condition d'arrêt sentinelle sans l'incorporer dans le calcul, filtre les valeurs aberrantes négatives et sécurise le bloc arithmétique final contre le crash critique de division par zéro (ZeroDivisionError) dans le cas où l'utilisateur saisit -1 dès la première entrée.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la valeur logique résultant de l'évaluation de l'expression : NON((5 > 2) ET (3 == 4)) ?

R : Vrai (True). L'expression interne (5 > 2) est Vraie, mais (3 == 4) est Fausse. La conjonction (Vrai ET Faux) s'évalue donc à Faux. L'application de l'opérateur de négation NON devant l'expression inverse le résultat : NON(Faux) donne Vrai.

Q : Quel est le risque algorithmique majeur lié à l'omission de la mise à jour des variables de contrôle dans le corps d'une boucle 'Tant Que' ?

R : Le non-respect de la terminaison algorithmique menant à une boucle infinie. Si la condition de continuation reste continuellement Vraie sans que son variant de boucle ne décroisse vers la condition d'arrêt, le programme monopolise les ressources du processeur et bloque l'exécution indéfiniment.

Q : Pourquoi l'utilisation de l'opérateur d'égalité stricte '==' pour comparer deux variables réelles à virgule flottante (Float) est-elle considérée comme une faute éliminatoire en examen d'algorithmique ?

R : En raison de l'imprécision inhérente à la représentation binaire des nombres réels en machine (norme IEEE 754), certains nombres décimaux finis ne peuvent pas être codés de manière exacte et subissent des erreurs d'arrondi infinitésimales (ex: 0.1 + 0.2 = 0.30000000000000004). Deux réels théoriquement égaux en mathématiques peuvent être distincts en mémoire. La comparaison correcte exige de vérifier que la valeur absolue de leur différence est inférieure à un seuil de tolérance infinitésimal epsilon : |a - b| < EPSILON.